

**DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING**

**21CS502-Computer Network and
Communication Lab Manual**

5th Semester, B.E.

Submitted By:

LESSON PLAN

Prerequisite:

- Sound knowledge in C/C++/Python, Linux
- Basic knowledge in Computer Networks

Objectives:

To provide a good understanding of the salient features of network Programming. Students are expected to understand the network concept in detail and to program using C/C++/Python programming language.

Learning outcome and end use:

By the end of the course, the students should be able to :-

- Have a detailed understanding of the underlying design principles of computers communication.
- Be able to apply basic concept of TCP/IP protocol and the methods of programming techniques.
- Recognize the features of client/server systems and programs with a view to be able to implement simple system in this model
- Analyse, develop and implement error-detection and correction, and encryption algorithms
- Be able to simulate network protocols to check the functionality of the protocols.

Rules of behavior in the laboratory:

- Every laboratory sessions begins SHARP at the specified time in the schedule.
- Each lab session is two hours long. Students are advised to bring their record and observation books.
- Strict discipline should be maintained throughout the lab.
- Punctuality should be followed by each and every student.
- Food, drinks and cell phone are not allowed inside the laboratory

Method of Assessment:

Write up	-	10
Program Execution	-	30
Viva Voce	-	10

List of Experiments

Note: Student is required to solve one problem from PART-A and one problem from PART-B.

PART A – Simulation Experiments

The following experiments shall be conducted using either NS2/OPNET or any other simulators.

1. Simulate a three nodes point-to-point network with duplex links between them. Set the queue size vary the bandwidth and find the number of packets dropped.
2. Simulate a four-node point-to-point network, and connect the links as follows: n0-n2, n1-n2 and n2-n3. Apply TCP agent between n0-n3 and UDP n1-n3. Apply relevant applications over TCP and UDP agents changing the parameter and determine the number of packets by TCP/UDP and analyse the throughput.
3. Simulate an Ethernet LAN using N-nodes (6-10), change data rate and compare the throughput.
4. Simulate an Ethernet LAN using N nodes and set multiple traffic nodes and determine collision across different nodes, also plot congestion window for different source/destination.

PART B

The following experiments shall be conducted using C/C++/Python.

1. Program to implement Bit Stuffing or Byte Stuffing concept in data link layer
2. Write a program for error detecting code using CRC-CCITT (16-bits).
3. Write a program for frame sorting technique used in buffers.
4. Write a program for distance vector algorithm to find suitable path for transmission.
5. Using TCP/IP sockets, write a client-server program to make client sending the file name and the server to send back the contents of the requested file if present.
6. Using UDP SOCKETS, write a client-server program to make the client sending two numbers and an operator, and server responding with the result. Display the result and appropriate messages for invalid inputs at the client side.

Part A Experiments

1. Overview of NS-2

Introduction to NS-2

NS-2 is an event driven packet level network simulator developed as part of the VINT project (Virtual Internet Testbed). This was a collaboration of many institutes including UC Berkeley, AT&T, XEROX PARC and ETH. Version 1 of NS was developed in 1995 and with version 2 released in 1996. Version 2 included a scripting language called Object oriented Tcl (OTcl). It is an open source software package available for both Windows and Linux platforms. It provides substantial support for simulation of TCP, routing, and multicast protocols over wired and wireless (local and satellite) networks.

NS-2 has many and expanding uses including:

- To evaluate the performance of existing network protocols.
- To evaluate new network protocols before use.
- To run large scale experiments not possible in real experiments.
- To simulate a variety of ip networks

Downloading/Installing ns

You can download the package from <http://www.isi.edu/nsnam/ns/ns-build.html>. There are two ways to build ns: from the various packages or 'all-in-one' package. For simplicity, it is recommended to start with the 'all-in-one' package. Please refer <http://www.isi.edu/nsnam/ns/ns-problems.html> for any installation problems.

Starting ns

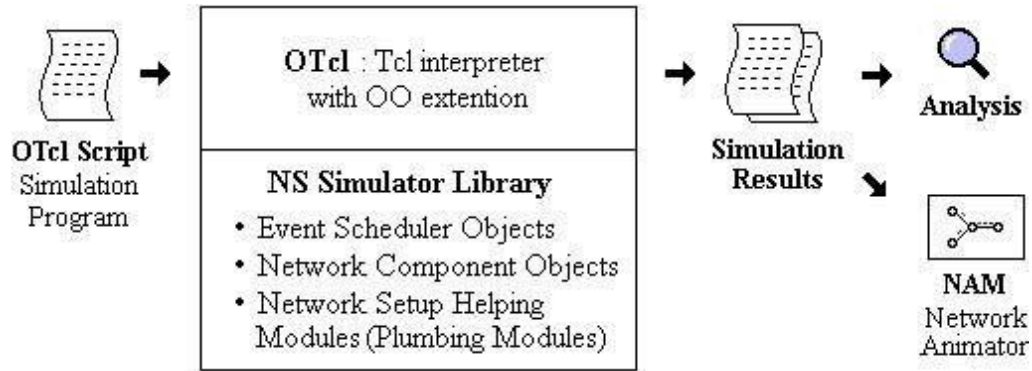
You start ns with the command 'ns <tclscript>' (assuming that you are in the directory with the ns executable, or that your path points to that directory), where '<tclscript>' is the name of a Tcl (Tool Command Language) script file which defines the simulation scenario (i.e. the topology and the events). You can also just start ns without any arguments and enter the Tcl commands in the Tcl shell, but that is definitely less comfortable.

Starting nam (Network Animator)

You can either start nam with the command 'nam <nam-file>' where '<nam-file>' is the name of a nam trace file that was generated by ns, or you can execute it directly out of the Tcl simulation script for the simulation which you want to visualize.

Architecture of NS-2

As shown in the simplified user's view of Figure, NS is an Object-oriented Tcl (OTcl) script interpreter that has a simulation event scheduler and network component object libraries, and network set-up (plumbing) module libraries.



An OTcl script will do the following.

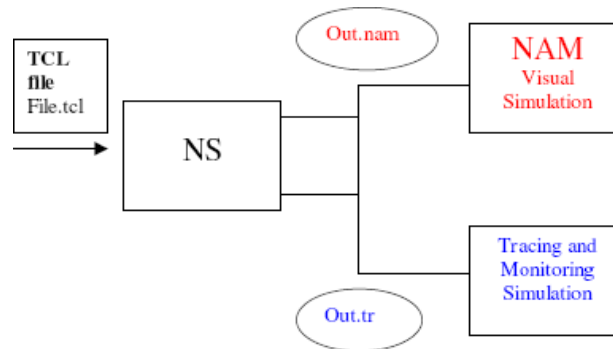
- Initiates an event scheduler.
- Sets up the network topology using the network objects.
- Tells traffic sources when to start/stop transmitting packets through the event scheduler

Another major component of NS besides network objects is the event scheduler. An event in NS is a packet ID that is unique for a packet with scheduled time and the pointer to an object that handles the event. The event scheduler in NS-2 performs the following tasks:

- Organizes the simulation timer.
- Fires events in the event queue.
- Invokes network components in the simulation.

Depending on the user's purpose for an OTcl simulation script, simulation results are stored as trace files, which can be loaded for analysis by an external application:

1. A NAM trace file (file.nam) for use with the Network Animator Tool
2. A Trace file (file.tr) for use with XGraph or TraceGraph [11].



TclCL is the language used to provide a linkage between C++ and OTcl. Toolkit Command Language (Tcl/OTcl) scripts are written to set up/configure network topologies. TclCL provides linkage for class hierarchy, object instantiation, variable binding and command dispatching. OTcl is used for periodic or triggered events.

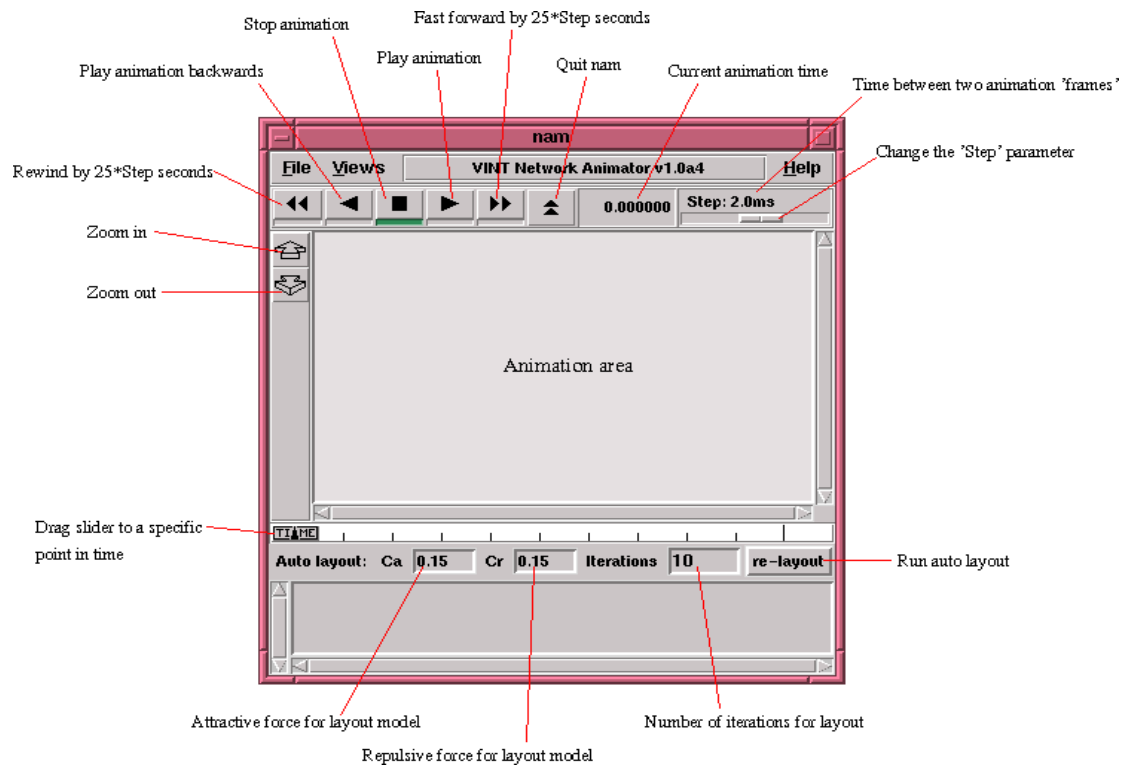
NS-2 features

NS-2 implements the following features

1. Router queue Management Techniques DropTail, RED, CBQ,
2. Multicasting
3. Simulation of wireless networks
 - Developed by Sun Microsystems + UC Berkeley (Daedalus Project)
 - Terrestrial (cellular, adhoc, GPRS, WLAN, BLUETOOTH), satellite
 - IEEE 802.11 can be simulated, Mobile-IP, and adhoc protocols such as
 - DSR, TORA, DSDV and AODV.
4. Traffic Source Behavior- www, CBR, VBR
5. Transport Agents- UDP/TCP
6. Routing
7. Packet flow
8. Network Topology
9. Applications- Telnet, FTP, Ping
10. Tracing Packets on all links/specific links

NAM (Network Animator)

NAM provides a visual interpretation of the network topology created. Below you can see a screenshot of a nam window where the most important functions are being explained.



Its features are as follows:

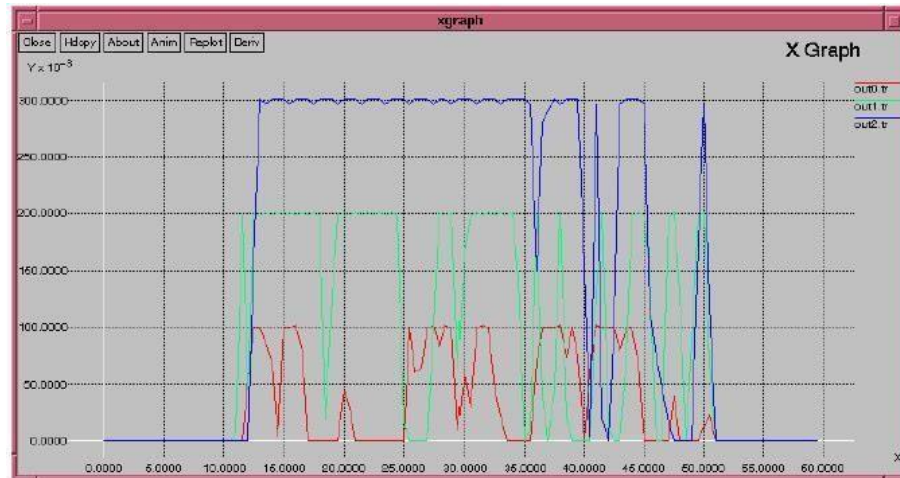
- Provides a visual interpretation of the network created
- Can be executed directly from a Tcl script
- Controls include play, stop ff, rw, pause, a display speed controller and a packet monitor facility.
- Presents information such as throughput, number packets on each link.
- Provides a drag and drop interface for creating topologies.

XGraph

XGraph is an X-Windows application that includes:

- Interactive plotting and graphing
- Animation and derivatives

To use XGraph in NS-2 the executable can be called within a TCL Script. This will then load a graph displaying the information visually displaying the information of the trace file produced from the simulation.



XGraph running comparing three trace files in a graph

TraceGraph

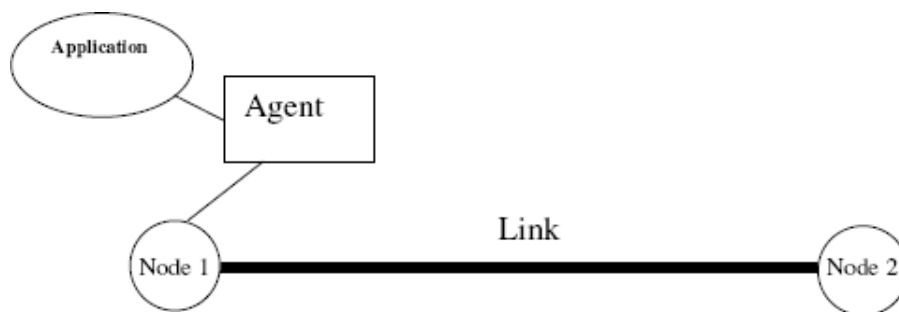
TraceGraph is a trace file analyzer that runs under Windows, Linux and UNIX systems and requires Mat lab 6.0 or higher.

TraceGraph supports the following trace file formats.

- Wired
- Satellite
- Wireless

2. OTcl Scripting with NS-2

In NS-2, the network is constructed using nodes which are connected using links. Events are scheduled to pass between nodes through the links. Nodes and links can have various properties associated with them. Agents can be associated with nodes and they are responsible for generating different packets (e.g. TCP agent or UDP agent). The traffic source is an application which is associated with a particular agent (e.g. ping application).



This diagram shows two nodes, a link, an agent and an application.

How to start?

First of all, you need to create a simulator object. This is done with the command

```
set ns [new Simulator]
```

Now we open a file for writing that is going to be used for the nam trace data.

```
set nf [open out.nam w]
$ns namtrace-all $nf
```

The first line opens the file 'out.nam' for writing and gives it the file handle 'nf'. In the second line we tell the simulator object that we created above to write all simulation data that is going to be relevant for nam into this file. The next step is to add a 'finish' procedure that closes the trace file and starts nam.

```
proc finish {} {
    global ns nf
    $ns flush-trace
    close $nf
    exec nam out.nam &
    exit 0
}
```

The last line finally starts the simulation

```
$ns run
```

Node creation and linking

The following two lines define the two nodes. (Note: You have to insert the code in this section **before** the line '\$ns run', or even better, before the line '\$ns at 5.0 "finish"').

```
set n0 [$ns node]
set n1 [$ns node]
```

A new node object is created with the command '\$ns node'. The above code creates two nodes and assigns them to the handles 'n0' and 'n1'. The next line connects the two nodes.

```
$ns duplex-link $n0 $n1 1Mb 10ms DropTail
```

This line tells the simulator object to connect the nodes n0 and n1 with a duplex link with the bandwidth 1Megabit, a delay of 10ms and a DropTail queue.

Now you can save your file and start the script with 'ns example1.tcl'. nam will be started automatically and you should see an output that resembles the picture below.

Sending data

The next step is to send some data from node n0 to node n1. In ns, data is always being sent from one 'agent' to another. So the next step is to create an agent object that sends data from node n0, and another agent object that receives the data on node n1.

```
#create a UDP agent and attach it to node n0
set udp0 [new Agent/UDP]
$ns attach-agent $n0 $udp0
# create a CBR traffic source and attach it to udp0
set cbr0 [new Application/Traffic/CBR]
$cbr0 set packetSize_ 500
$cbr0 set interval_ 0.005
$cbr0 attach-agent $udp0
```

These lines create a UDP agent and attach it to the node n0, then attach a CBR traffic generator to the UDP agent. CBR stands for 'constant bit rate'. Line 7 and 8 should be self-explaining. The packet Size is being set to 500 bytes and a packet will be sent every 0.005 seconds (i.e. 200 packets per second). The next lines create a Null agent which acts as traffic sink and attach it to node n1.

```
set null0 [new Agent/Null]
$ns attach-agent $n1 $null0
```

Now the two agents have to be connected with each other.

```
$ns connect $udp0 $null0
```

And now we have to tell the CBR agent when to send data and when to stop sending. It's probably best to put the following lines just before the line '\$ns at 5.0 "finish"'.

```
$ns at 0.5 "$cbr0 start"
$ns at 4.5 "$cbr0 stop"
```

Now you can save the file and start the simulation again. When you click on the 'play' button in the nam window, you will see that after 0.5 simulation seconds, node 0 starts sending data packets to node 1. You might want to slow nam down then with the 'Step' slider.

Add the following two lines to your CBR agent definitions.

```
$udp0 set class_ 1
$udp1 set class_ 2
```

The parameter 'fid_' stands for 'flow id'.

Now you can add the following piece of code to your Tcl script, preferably at the beginning after the simulator object has been created, since this is a part of the simulator setup.

```
$ns color 1 Blue
$ns color 2 Red
```

This code allows you to set different colors for each flow id.

You can add the following line to your code to monitor the queue for the link from n2 to n3.

```
$ns duplex-link-op $n2 $n3 queuePos 0.5
```

You can see the packets in the queue now, and after a while you can even see how the packets are being dropped, though (at least on my system, I guess it might be different in later or earlier releases) only blue packets are being dropped. But you can't really expect too much 'fairness' from a simple Drop Tail queue. So let's try to improve the queuing by using a SFQ (stochastic fair queuing) queue for the link from n2 to n3. Change the link definition for the link between n2 and n3 to the following line.

```
$ns duplex-link $n3 $n2 1Mb 10ms SFQ
```

The queuing should be 'fair' now. The same amount of blue and red packets should be dropped.

Experiment No. 1

Problem Statement:

Simulate a three nodes point-to-point network with duplex links between them. Set the queue size vary the bandwidth and find the number of packets dropped.

```
#File Name: ns_expl.tcl
#Description: Simulating simple three nodes point-to-point network
#####

set ns [new Simulator]

#Open a new file for NAMTRACE
set nf [open out.nam w]
$ns namtrace-all $nf

#Open a new file to log TRACE
set tf [open out.tr w]
$ns trace-all $tf

#Body of the finish procedure
proc finish {} {
    global ns nf tf
    $ns flush-trace
    close $nf
    close $tf
    exec nam out.nam &
    exit 0
}

#Create Nodes
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]

#Create Links between Nodes
$ns duplex-link $n0 $n1 1Mb 10ms DropTail
$ns duplex-link $n1 $n2 1Mb 10ms DropTail

#Set the queue limit - default is 50 packets
$ns queue-limit $n0 $n1 50
$ns queue-limit $n1 $n2 50

#Create Transport Agent
set udp0 [new Agent/UDP]
$ns attach-agent $n0 $udp0
set null0 [new Agent/Null]
$ns attach-agent $n2 $null0
$ns connect $udp0 $null0

#Create Application to generate traffic
set cbr0 [new Application/Traffic/CBR]
$cbr0 set packetSize_ 500
$cbr0 set interval_ 0.005
$cbr0 attach-agent $udp0

#Start and Stop generating traffic
```

```
$ns at 0.5 "$cbr0 start"
$ns at 4.5 "$cbr0 stop"
```

```
#Stop the simulation
$ns at 5.0 "finish"
```

```
#Run the simulation
$ns run
```

Nam output:



Experiment No. 2

Problem Statement:

Simulate a four node point-to-point network, and connect the links as follows: n0-n2, n1-n2 and n2-n3. Apply TCP agent between n0-n3 and UDP n1-n3. Apply relevant applications over TCP and UDP agents changing the parameter and determine the number of packets by TCP/UDP and analyze the throughput..

```
#File Name: ns_exp2.tcl
```

```
#Description: Simulating four node point-to-point network with TCP and UDP agent
#####
```

```
set ns [new Simulator]
```

```
#Open a new file for NAMTRACE
set nf [open out.nam w]
$ns namtrace-all $nf
```

```
#Open a new file to log TRACE
set tf [open out.tr w]
$ns trace-all $tf
```

```
#Body of the 'finish' procedure
proc finish {} {
    global ns nf tf
    $ns flush-trace
    close $nf
    close $tf
}
```

```

        exec nam out.nam &
        exit 0
    }

#Create Nodes
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]

#Create Links between Nodes
$ns duplex-link $n0 $n2 1Mb 10ms DropTail
$ns duplex-link $n1 $n2 1Mb 10ms DropTail
$ns duplex-link $n2 $n3 1Mb 10ms DropTail

#Set the queue limit - default is 50 packets
$ns queue-limit $n0 $n2 50
$ns queue-limit $n1 $n2 50
$ns queue-limit $n2 $n3 50

#Create TCP Agent between node 0 and node 3
set tcp0 [new Agent/TCP]
$ns attach-agent $n0 $tcp0
set sink0 [new Agent/TCPSink]
$ns attach-agent $n3 $sink0
$ns connect $tcp0 $sink0

#Create FTP Application for TCP Agent
set ftp0 [new Application/FTP]
$ftp0 attach-agent $tcp0

#Specify TCP packet size
Agent/TCP set packetSize_ 1000

#Create UDP Agent between node 1 and node 3
set udp0 [new Agent/UDP]
$ns attach-agent $n1 $udp0
set null0 [new Agent/Null]
$ns attach-agent $n3 $null0
$ns connect $udp0 $null0

#Create CBR Application for UDP Agent
set cbr0 [new Application/Traffic/CBR]
$cbr0 set packetSize_ 500
$cbr0 set interval_ 0.005
$cbr0 attach-agent $udp0

#Start and Stop FTP Traffic
$ns at 0.75 "$ftp0 start"
$ns at 4.75 "$ftp0 stop"

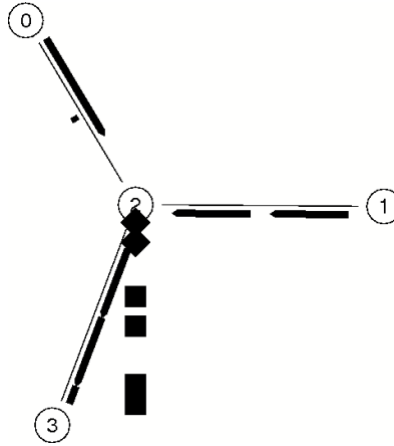
#Start and Stop CBR traffic
$ns at 0.5 "$cbr0 start"
$ns at 4.5 "$cbr0 stop"

#Stop the simulation
$ns at 5.0 "finish"

```

```
#Run the simulation
$ns run
```

NAM output:



Experiment No. 3

Problem Statement:

Simulate an Ethernet LAN using N-nodes (6-10), change error rate and data rate and compare the throughput.

```
#File Name: ns_exp5.tcl
#Description: Simulate an Ethernet LAN using N-nodes (6 10)
#####

set ns [new Simulator]

#Open a new file for NAMTRACE
set nf [open out.nam w]
$ns namtrace-all $nf

#Open a new file to log TRACE
set tf [open out.tr w]
$ns trace-all $tf

#Body of the finish procedure
proc finish {} {
    global ns nf tf
    $ns flush-trace
    close $nf
    close $tf
    exec nam out.nam &
    exit 0
}
```

```

#Create Nodes
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]
set n4 [$ns node]
set n5 [$ns node]
set n6 [$ns node]
set n7 [$ns node]
set n8 [$ns node]
set n9 [$ns node]
set n10 [$ns node]

#Create a Local Area Network (LAN) of 10 Nodes
$ns make-lan "$n0 $n1 $n2 $n3 $n4 $n5 $n6 $n7 $n8 $n9 $n10" 100Mb
10ms LLQueue/DropTail Mac/802_3

#Create TCP Agent between node 0 and node 3
set tcp0 [new Agent/TCP]
$ns attach-agent $n0 $tcp0
set sink0 [new Agent/TCPSink]
$ns attach-agent $n3 $sink0
$ns connect $tcp0 $sink0

#Create FTP Application for TCP Agent
set ftp0 [new Application/FTP]
$ftp0 attach-agent $tcp0

#Specify TCP packet size
Agent/TCP set packetSize_ 1000

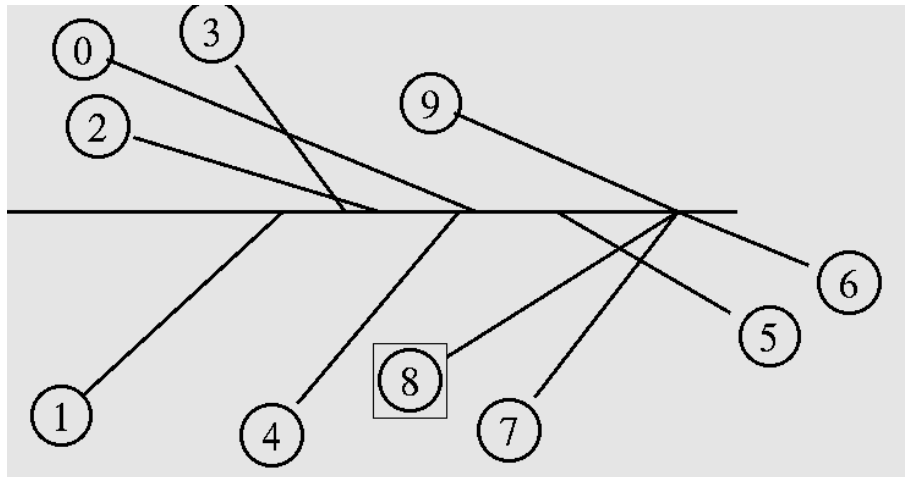
#Start and Stop FTP Traffic
$ns at 0.75 "$ftp0 start"
$ns at 4.75 "$ftp0 stop"

#Stop the simulation
$ns at 5.0 "finish"

#Run the simulation
$ns run

```

NAM output:



Experiment No. 4

Problem Statement:

Simulate an Ethernet LAN using N nodes and set multiple traffic nodes and determine collision across different nodes.

```
#File Name: ns_exp6.tcl

#Description: Simulate an Ethernet LAN and set multiple traffic nodes
#####

set ns [new Simulator]

#Open a new file for NAMTRACE
set nf [open out.nam w]
$ns namtrace-all $nf

#Open a new file to log TRACE
set tf [open out.tr w]
$ns trace-all $tf

#Body of the 'finish' procedure
proc finish {} {
    global ns nf tf
    $ns flush-trace
    close $nf
    close $tf
    exec nam out.nam &
    exit 0
}

#Create Nodes
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]
set n4 [$ns node]
set n5 [$ns node]
set n6 [$ns node]
set n7 [$ns node]
set n8 [$ns node]
set n9 [$ns node]

#Create a Local Area Network (LAN) of 10 Nodes
$ns make-lan "$n0 $n1 $n2 $n3 $n4 $n5 $n6 $n7 $n8 $n9" 100Mb LL
Queue/DropTail Mac/802_3

#Create TCP Agent between node 0 and node 3
set tcp0 [new Agent/TCP]
$ns attach-agent $n0 $tcp0
```

```
set sink0 [new Agent/TCPSink]
$ns attach-agent $n3 $sink0
$ns connect $tcp0 $sink0

#Create FTP Application for TCP Agent
set ftp0 [new Application/FTP]
$ftp0 attach-agent $tcp0

#Specify TCP packet size
Agent/TCP set packetSize_ 1000

#Create TCP Agent between node 1 and node 3
set tcp1 [new Agent/TCP]
$ns attach-agent $n1 $tcp1
set sink1 [new Agent/TCPSink]
$ns attach-agent $n3 $sink1
$ns connect $tcp1 $sink1

#Create Telnet Application for TCP Agent
set telnet0 [new Application/Telnet]
$telnet0 set interval_ 0.005
$telnet0 attach-agent $tcp1

#Start and Stop FTP Traffic
$ns at 0.75 "$ftp0 start"
$ns at 4.75 "$ftp0 stop"

#Start and Stop Telnet traffic
$ns at 0.5 "$telnet0 start"
$ns at 4.5 "$telnet0 stop"

#Stop the simulation
$ns at 5.0 "finish"

#Run the simulation
$ns run
```

Experiment No. 5**Problem Statement:**

Simulate an Ethernet LAN using N nodes and set multiple traffic nodes and plot congestion window for different source/destination.

```
#File Name: ns_exp7.tcl

#Description: Simulate an Ethernet LAN using N nodes and set multiple traffic
nodes and plot congestion window for different source/destination.
#####

set ns [new Simulator]

#Open a new file for NAMTRACE
set nf [open out.nam w]
$ns namtrace-all $nf

#Open a new file to log TRACE
set tf [open out.tr w]
$ns trace-all $tf

#Body of the 'finish' procedure
proc finish {} {
    global ns nf tf
    $ns flush-trace
    close $nf
    close $tf
    exec nam out.nam &
    exit 0
}

#Create Nodes
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]
set n4 [$ns node]
set n5 [$ns node]
set n6 [$ns node]
set n7 [$ns node]
set n8 [$ns node]
set n9 [$ns node]

#Create a Local Area Network (LAN) of 10 Nodes
$ns make-lan "$n0 $n1 $n2 $n3 $n4 $n5 $n6 $n7 $n8 $n9" 100Mb LL
Queue/DropTail Mac/802_3

#Create TCP Agent between node 0 and node 3
set tcp0 [new Agent/TCP]
$ns attach-agent $n0 $tcp0
set sink0 [new Agent/TCPSink]
$ns attach-agent $n3 $sink0
```

```

$ns connect $tcp0 $sink0

#Open a new file to log Congestion Window data
set cfile0 [open tcp0.tr w]
$tcp0 attach $cfile0
$tcp0 trace cwnd_

#Create FTP Application for TCP Agent
set ftp0 [new Application/FTP]
$ftp0 attach-agent $tcp0

#Specify TCP packet size
Agent/TCP set packetSize_ 1000

#Create TCP Agent between node 1 and node 3
set tcp1 [new Agent/TCP]
$ns attach-agent $n1 $tcp1
set sink1 [new Agent/TCPSink]
$ns attach-agent $n3 $sink1
$ns connect $tcp1 $sink1

#Open a new file to log Congestion Window data
set cfile1 [open tcp1.tr w]
$tcp1 attach $cfile1
$tcp1 trace cwnd_

#Create Telnet Application for TCP Agent
set telnet0 [new Application/Telnet]
$telnet0 set interval_ 0.005
$telnet0 attach-agent $tcp1

#Start and Stop FTP Traffic
$ns at 0.75 "$ftp0 start"
$ns at 4.75 "$ftp0 stop"

#Start and Stop Telnet traffic
$ns at 0.5 "$telnet0 start"
$ns at 4.5 "$telnet0 stop"

#Stop the simulation
$ns at 5.0 "finish"

#Run the simulation
$ns run

```

Part B Programs

Experiment No. 1

Problem Statement:

Program to implement Bit Stuffing or Byte Stuffing concept in data link layer

```
def sender():
    n = int(input("Enter the no. of bits: ")) print("Enter", n, "bits:")
    data = [int(input()) for _ in range(n)] add = [0, 1, 1, 1, 1, 1, 1, 0]
    frame = [] frame += add count = 0
    for i in data:
        if count == 5:
            frame.append(0) count = 0
        frame.append(i) if i == 1:
            count += 1 frame += add
    print("Sent Frame:", frame) receiver(frame)
```

```
def receiver(frame): data = []
count = 0
```

```
for i in frame[8:-8]:
    if i==1:
        data.append(i) count+=1
    if i==0:
        if count==5:
            count=0 else:
                data.append(i)
```

```
print("Data Received is:", data)
```

```
sender()
```

Experiment No. 2

Problem Statement:

Write a program for error detecting code using CRC-CCITT (16-bits).

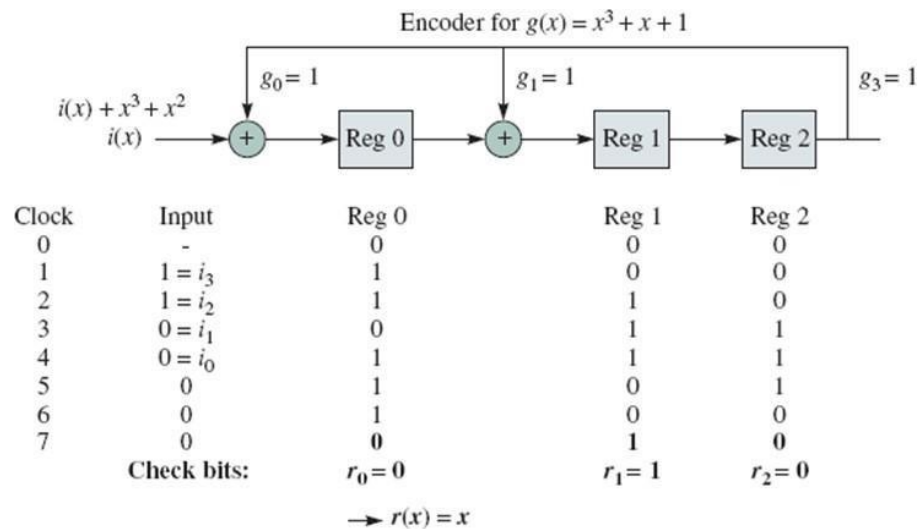
Theory:

It does error checking via polynomial division. In general, a bit string can be represented using a polynomial. Ex: 10010101110 can be represented as $X^{10} + X^7 + X^5 + X^3 + X^2 + X$. All computations are done in modulo 2. The algorithm uses $G(x) = X^{16} + X^{12} + X^5 + 1$ as the generator polynomial which used in HDLC, X.25, V.41, XMODEM, Bluetooth, SD and many others.

Algorithm:

1. Given a bit string, append 0s to the end of it (the number of 0s is the same as the degree of the generator polynomial). Let $B(x)$ be the polynomial corresponding to B.
2. Divide $B(x)$ by some agreed on polynomial $G(x)$ (generator polynomial) and determine the remainder $R(x)$. This division is to be done using Modulo 2 Division.
3. Define $T(x) = B(x) - R(x)$, ($T(x)/G(x) \Rightarrow$ remainder 0)
4. Transmit T, the bit string corresponding to $T(x)$.
5. Let T' represent the bit stream the receiver gets and $T'(x)$ the associated polynomial. The receiver divides $T'(x)$ by $G(x)$. If there is a 0 remainder, the receiver concludes $T = T'$ and no error occurred otherwise, the receiver concludes an error occurred and requires a retransmission.

The algorithm can be implemented using a feedback shift-register as shown below.



```

    /*****
*   File Name    : crc.c
*
*   Description :Program for error detecting code using CRC-CCITT (16-bits).
*****/

#include<stdio.h>

#define MAX_SIZE  20

//this structure 16 bit CRC
struct reg
{
    int  bit;
}r[16];

int n; // no. of input bits
int input[MAX_SIZE];

// function to implement the encoder for CCITT-16 polynomial

void compute_crc()
{
    int i,j;
    int lmb; //corresponds to right most bit of the  shifft-register circuit.

    for(j=0;j<(n+16);j++)
    {
        lmb=r[15].bit;

        // shift the bits
        for(i=15;i>0;i--)
        {
            r[i].bit=r[i-1].bit;
        }

        r[0].bit=input[j];

        //if leftmost bit is 1 , XOR the Dividend and Divisor.
        //you can make your program portable by avoiding ^ operator
        if(lmb==1)
        {
            r[12].bit = r[12].bit ^ lmb;
            r[5].bit = r[5].bit ^ lmb;
            r[0].bit = r[0].bit ^ lmb;
        }
    }
    printf("Register content:\n");

    for(i=15;i>=0;i--)
        printf("%d ",r[i].bit);

    for(i=n,j=15;j>=0;i++,j--)
        input[i]=r[j].bit;
}

```



```

// function to check error in the transmitted data

int check_err()
{
    int i;

    for(i=15;i>=0;i--)
    {
        if(r[i].bit != 0)
        {
            return i;
        }
    }
    return 0;
}

void read_input()
{
    int i;
    printf("\nEnter the number of bits in the input:\n");
    scanf("%d",&n);

    printf("\nEnter the information bits(1's & 0's):\n");
    for(i=0;i<n;i++)
        scanf("%d", &input[i]);

    for(i=n;i<(n+16);i++)
        input[i]=0;
}

int main()
{
    int i;

    read_input();

    printf("\n\nAt sender:\n\n");

    //initialize the registers
    for(i=0;i<16;i++)
        r[i].bit=0;

    compute_crc();

    printf("\nThe total message along with crc :\n");
    for(i=0;i<(n+16);i++)
        printf("%d ",input[i]);

    printf("\n\nThe data is transmitted\n");

    printf("\n\nAt receiver:\n\n");
    printf("\nEnter the received data \n");
    for(i=0;i<(n+16);i++)
        scanf("%d", &input[i]);
}

```

```

//initialize the registers
for(i=0;i<16;i++)
    r[i].bit=0;

// compute CRC
compute_crc();

// check for errors
if(i=check_err())
    printf("\nThere is error in the received data. Error position=%d",i);
else
    printf("\nThe received data is fine");

return 0;
}

```

OR

Python Code for CRC-16

```

div=[1,0,0,0,1,0]
n=int(input("Enter the number of bits : ")) data= [int(input()) for _ in range(n)] m=len(div)
if n<m:
    print("Invalid data ") else:
cdata=data.copy() for i in range(m-1):
cdata.append(0)

for i in range(n):
    if cdata[i]==1:
        for j in range(m):
            cdata[i+j]^=div[j]

for i in range(n,n+m-1):
    data.append(cdata[i])

print("Data to be transmitted is :",data)

ch=input("Introduce Error - Y->YES, N->NO : ") if ch=="Y":
p=int(input("Enter the position to change bit : ")) if p-1<0 or p-1>n+m-2:
print("Invalid Position!!") else:
if data[p-1]==1:
    data[p-1]=0
else:
    data[p-1]=1

print("The message received is :",data)

for i in range(n):
    if data[i]==1:
        for j in range(m):

```

```

data[i+j]^=div[j]

flag=True

for i in data:
    if i==1:
        flag=False break

if(flag):
    print("No Error\nSucessfully Transferred ")
else:
    print("Error!!")

```

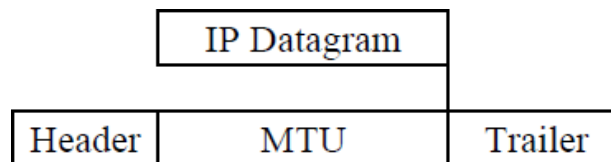
Experiment No. 3

Problem Statement:

Write a program for frame sorting technique used in buffers.

Theory:

The data link layer divides the stream of bits received from the network layer into manageable data units called frames. If frames are to be distributed to different systems on the network, the Data link layer adds a header to the frame to define the sender and/or receiver of the frame. Each Data link layer has its own frame format. One of the fields defined in the format is the maximum size of the data field. In other words, when datagram is encapsulated in a frame, the total size of the datagram must be less than this maximum size, which is defined by restriction imposed by the hardware and software used in the network.



The value of MTU differs from one physical network to another. In order to make IP protocol portable/independent of the physical network, the packagers decided to make the maximum length of the IP datagram equal to the largest Maximum Transfer Unit (MTU) defined so far. However, for other physical networks we must divide the datagrams to make it possible to pass through these networks.

This is called fragmentation. When a datagram is fragmented, each fragmented has its own header. A fragmented datagram may itself be fragmented if it encounters a network with an even smaller MTU. In another words, a datagram may be fragmented several times before it reached the final destination and also, the datagrams referred to as (frames in Data link layer) may arrives out of order at destination. Hence sorting of frames need to be done at the destination to recover the original data. The following program simulates the frame sorting technique used in buffers.

Algorithm

1. Read a message from the keyboard
2. Divide the message into fixed size packets. Include sequence number and other information in each packet.
3. Shuffle the packets (to generate out of order packets)
4. Order the packets based on sequence numbers
5. Display the ordered packets.

```

/*****
*   File Name    : FrameSort.c
*
*   Description  : A program for frame sorting technique used in buffers
*****/

#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <time.h>

#define DATA_SZ 3

typedef struct packet
{
    int SeqNum;
    char Data[DATA_SZ+1];
} PACKET;

PACKET *readdata, *transdata;
time_t t ;

// Breaks the message into packets
int divide(char *msg)
{
    int msglen, NoOfPacket, i, j;
    msglen = strlen(msg);
    NoOfPacket = msglen/DATA_SZ;

    if ((msglen%DATA_SZ) !=0)
        NoOfPacket++;

```

```

readdata = (PACKET *)malloc(sizeof(PACKET) * NoOfPacket);
for(i = 0; i < NoOfPacket; i++)
{
    readdata[i].SeqNum = i + 1;
    for (j = 0; (j < DATA_SZ) && (*msg != '\0'); j++, msg++)
        readdata[i].Data[j] = *msg;
    readdata[i].Data[j] = '\0';
}

printf("\nThe Message has been divided as follows\n");
printf("\nPacket No.\tData\n\n");
for (i = 0; i < NoOfPacket; i++)
    printf(" %d\t\t%s\n", readdata[i].SeqNum, readdata[i].Data);

return NoOfPacket;
}

// shuffles the packets
void shuffle(int NoOfPacket)
{
    int *Status;
    int i, j, trans;

    srand(time(&t)); //every time you shuffle,get different random sequence
    Status=(int *)calloc(NoOfPacket, sizeof(int));
    transdata = (PACKET *)malloc(sizeof(PACKET) * NoOfPacket);

    for (i = 0; i < NoOfPacket;)
    {
        trans = rand()%NoOfPacket;
        if (Status[trans]!=1)
        {
            transdata[i].SeqNum = readdata[trans].SeqNum;
            strcpy(transdata[i].Data, readdata[trans].Data);
            i++;
            Status[trans] = 1;
        }
    }

    free(Status);
}

// sorts the packets
void sortframes(int NoOfPacket)
{
    PACKET temp;
    int i, j;

    for (i = 0; i < NoOfPacket; i++)
    {
        for (j = 0; j < NoOfPacket - (i+1); j++)
        {
            if (transdata[j].SeqNum > transdata[j + 1].SeqNum)
            {

```

```

        temp.SeqNum = transdata[j].SeqNum;
        strcpy(temp.Data, transdata[j].Data);
        transdata[j].SeqNum = transdata[j + 1].SeqNum;
        strcpy(transdata[j].Data, transdata[j + 1].Data);
        transdata[j + 1].SeqNum = temp.SeqNum;
        strcpy(transdata[j + 1].Data, temp.Data);
    }
}

// receives packets out of order and calls sort function
void receive(int NoOfPacket)
{
    int i;
    printf("\nPackets received in the following order\n");
    for (i = 0; i < NoOfPacket; i++)
        printf("%4d",transdata[i].SeqNum);

    sortframes(NoOfPacket);

    printf("\n\nPackets in order after sorting..\n");
    for (i = 0; i < NoOfPacket; i++)
        printf("%4d",transdata[i].SeqNum);
    printf("\n\nMessage received is :\n");
    for (i = 0; i < NoOfPacket; i++)
        printf("%s",transdata[i].Data);
}

int main()
{
    char msg[25];
    int NoOfPacket;
    printf("\nEnter The message to be Transmitted :\n");
    scanf("%[^\n]", msg);
    NoOfPacket = divide(msg);
    shuffle(NoOfPacket);
    receive(NoOfPacket);
    free(readdata);
    free(transdata);
    return 0;
}

```

OR

Python Code for Frame Sorting

```

import random
DATA_SZ=3

def bub_sort(arr):
    n=len(arr)
    for i in range(n):
        for j in range(n-i-1):
            if arr[j]>arr[j+1]:
                arr[j],arr[j+1]=arr[j+1],arr[j]

```

```
msg=input("Enter Message : ")

msg_chunks=[msg[i:i+DATA_SZ] for i in range(0,len(msg),DATA_SZ)]

frames=list(enumerate(msg_chunks,start=1))

print("Fragemented Frames : ",frames)

for i in range(len(frames)):
    j=random.randint(0,len(frames)-1)
    frames[i],frames[j]=frames[j],frames[i]

print("Shuffled Frames : ",frames)

bub_sort(frames)

print("Sorted Frames : ",frames)

print("Sorted Message : "+"".join(x[1] for x in frames))
```

Experiment No. 4

Problem Statement:

Write a program for Distance Vector Algorithm to find suitable path for transmission.

Theory:

The distance vector routing algorithm is sometimes called by other names, including the distributed Bellman-Ford routing algorithm and the Ford-Fulkerson algorithm, after the researchers who developed it. In distance vector routing, each router maintains a routing table indexed by, and containing one entry for, each router in subnet. This entry contains two parts: the preferred out going line to use for that destination, and an estimate of the time or distance to that destination. The metric used might be number of hops, time delay in milliseconds, total number of packets queued along the path, or something similar.

Algorithm:

1. Read the initial routing table. The initial routing table for each router consists of the distances to each of its neighbours. Distance is assumed to be infinity if there is no direct path.
2. To build the final routing table, each router communicates with each of its neighbours to update the routing table. The basic principle here is as follows:

Consider a node X that is interested in routing to destination Y via its directly attached neighbour Z. Node X's distance table entry, $D^X(Y, Z)$ is the sum of the cost of the direct link between X and Z, $c(X, Z)$, plus neighbour Z's currently known minimum cost path from itself (Z) to Y. That is:

$$D^X(Y, Z) = c(X, Z) + \min_w \{D^Z(Y, w)\}$$

The $\min_w$ term is taken over all of Z's directly attached neighbours (including X)

```

/*****
*   File Name      : DistVect.c
*
*   Description    : A program for distance vector algorithm to find suitable
                     path for transmission.
*****/

#include<stdio.h>

#define INFINITY 999

struct
{
    int cost;
    int via;

```



```

}routeTable[10][10];

int n; //no. of nodes

// function to build DV routing table
void build_route_table(int i)
{
    int j,k,new_cost=0;

    for(j=0;j<n;j++)
    {
        for(k=0;k<n && j!=i;k++)
        {

            // consider only adjacent routers
            if(routeTable[i][j].cost != INFINITY)
            {
                new_cost =routeTable[i][j].cost+routeTable[j][k].cost;
                if(routeTable[i][k].cost > new_cost)
                {
                    routeTable[i][k].cost=new_cost;
                    routeTable[i][k].via=routeTable[i][j].via;
                }
            }
        }
    }
}

// function to find shortest path between 2 routers
void find_path(int i,int j)
{
    printf("%c",'A' + i);
    if(i != j)
    {
        printf(" --> ");
        find_path(routeTable[i][j].via,j);
    }
}

// function to display routing table for each router
void disp_route_table(int i)
{
    int j;
    printf("\nFinal Routing Table for %c: ", 'A' + i);
    printf("\n\tDestination\tCost\tOutgoing line");
    printf("\n\t-----\t-----\t-----\n");
    for(j=0;j<n;j++)
    {
        printf("\n\t\t%c", 'A' + j);
        printf("\t%d",routeTable[i][j].cost);
        printf("\t%c", 'A' + routeTable[i][j].via);
        printf("\n");
    }
}

// function to read initial routing table for each router.
void read_route_table()

```

```
{
    int i,j;

    printf("Enter the initial routing table (if no direct node, enter 999):\n");
    for(i=0;i<n;i++)
    {
        printf("\nRouting table for %c:\n",'A' + i);
        for(j=0;j<n;j++)
        {
            if(i==j)
                routeTable[i][j].cost=0;
            else
            {
                printf("--> %c:", 'A' + j);
                scanf("%d",&routeTable[i][j].cost);
            }
            if(routeTable[i][j].cost != INFINITY)
                routeTable[i][j].via=j;
            else
                routeTable[i][j].via=INFINITY;
        }
    }
}
```

```

// main function

int main()
{
    int src,dst,i;
    int opt;

    printf("Enter the Number of routers:");
    scanf("%d",&n);

    //read initial routing table
    read_route_table();

    //build the routing table
    for(i=0;i<n;i++)
        build_route_table(i);

    //display the final routing table
    for(i=0;i<n;i++)
        disp_route_table(i);

    do
    {
        printf("\nEnter the Source node(0 to %d): ",n-1);
        scanf("%d",&src);
        printf("Enter the Destination node(0 to %d):",n-1);
        scanf("%d",&dst);

        if(src > (n - 1) || dst > (n - 1))
            printf("\n router doest not exist");
        else
        {
            find_path(src,dst);
            printf("\nThe cost of the shortest route
            is:\t%d\n",routeTable[src][dst].cost);
        }

        printf("\nDo you want to continue? (0/1):\n");
        scanf("%d",&opt);
    }while(opt);

    return 0;
}

```

Experiment No. 5

Problem Statement:

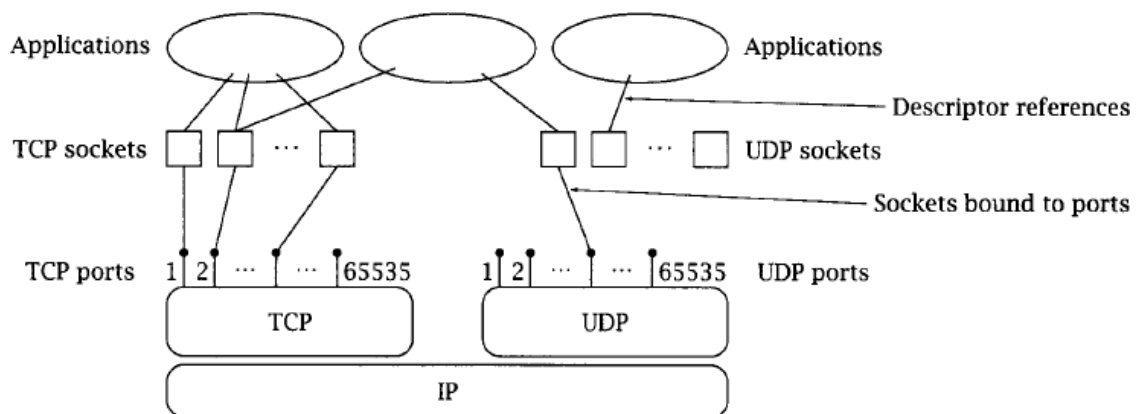
Using TCP sockets, write a client-server program to make client sending the file name and the server to send back the contents of the requested file if present.

Theory:

A socket allows an application to "plug in" to the network and communicate with other applications that are also plugged in to the same network. Information written to the socket by an application on one machine can be read by an application on a different machine, and vice versa. Sockets come in different flavors, corresponding to different underlying protocol families and different stacks of protocols within a family. we deal only with the TCP/IP protocol family. The main flavors of sockets in the TCP/IP family are stream sockets and datagram sockets. Stream sockets use TCP as the end-to-end protocol (with IP underneath) and thus provide a reliable byte-stream service. Datagram sockets use UDP (again, with IP underneath) and thus provide a best-effort datagram service that applications can use to send individual messages up to about 65,500 bytes in length.

A socket using the TCP/IP protocol family is uniquely identified by an Internet address, an end-to-end protocol (TCP or UDP), and a port number. When a socket is first created, it has an associated protocol but no Internet address or port number. Until a socket is bound to a port number, it cannot receive messages from a remote application.

Sockets, protocols, and ports.



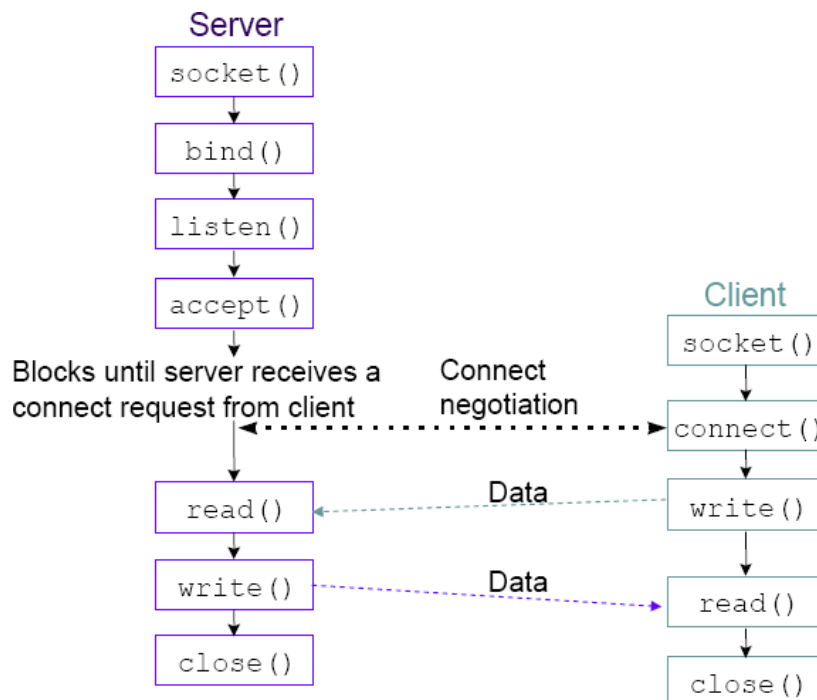
Algorithm (Client Side)

1. Start.
2. Create a socket using `socket ()` system call.
3. Connect the socket to the address of the server using `connect ()` system call.
4. Send the filename of required file using `send ()` system call.
5. Read the contents of the file sent by server by `recv ()` system call.
6. Stop.

Algorithm (Server Side)

1. Start.
2. Create a socket using `socket ()` system call.
3. Bind the socket to an address using `bind ()` system call.
4. Listen to the connection using `listen ()` system call.
5. Accept connection using `accept ()`
6. Receive filename and transfer contents of file with client.
7. Stop.

Client/Server communication using TCP socket:



```
#tcp_server.py

import
socket
import os

server_address = ('localhost', 8888)

server_socket = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
server_socket.bind(server_address)

server_socket.listen(1)
print("Server is listening for incoming connections...")

while True:
    client_socket, client_address =
server_socket.accept() print("Accepted connection
from", client_address)

    file_name =
client_socket.recv(1024).decode()
print("Received file name= ",
file_name)

    if os.path.exists(file_name):
        with open(file_name,
            'rb') as file:
            file_contents =
            file.read()
            client_socket.send(file_
contents) else:
            client_socket.send(b'File
not found')
    client_socket.close()
```

#tcp_client.py

```

import socket

server_address = ('localhost', 8888)

client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect(server_address)

file_name = input("Enter the file name: ")

client_socket.send(file_name.encode())

file_contents = client_socket.recv(1024).decode()

if file_contents == b'File not found':
    print("File not found on the server.")
else:
    print("File Contents :\n",file_contents)
    with open("received_" + file_name, 'w') as file:
        file.write(file_contents)
    print("File received and saved as 'received_" + file_name + "'")

client_socket.close()

```

Experiment No. 6**Problem Statement**

Using UDP SOCKETS, write a client-server program to make the client sending two numbers and an operator, and server responding with the result. Display the result and appropriate messages for invalid inputs at the client side.

Theory: Refer to the Experiment No. 4

Algorithm (Client Side)

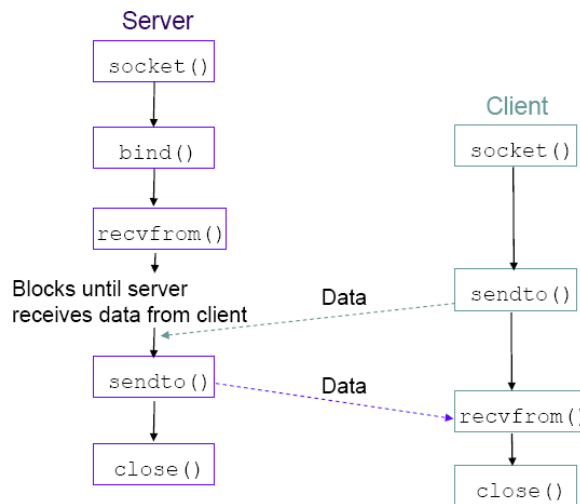
1. Start.
2. Create a socket using socket () system call.

3. Connect the socket to the address of the server using `connect ()` system call.
4. Read two numbers and an operator, and then using `sendto ()` system call.
5. Receive the result sent by server using `recvfrom ()` system call.
6. Stop.

Algorithm (Server Side)

1. Start.
2. Create a socket using `socket ()` system call.
3. Bind the socket to an address using `bind ()` system call.
4. Listen to the connection using `listen ()` system call.
5. Accept connection using `accept ()`
6. Receive the numbers and operator. Send the result to the client.
7. Stop.

Client/Server communication using TCP socket



#udp_server.py

```
import socket
import re
```

```
localIP = "127.0.0.1"
```

```
localPort = 20001
```



```
bufferSize = 1024
```

```
UDPServerSocket = socket.socket(family=socket.AF_INET, type=socket.SOCK_DGRAM)
```

```
UDPServerSocket.bind((localIP, localPort))
```

```
print("UDP server up and listening...")
```

```
while True:
```

```
    bytesAddressPair = UDPServerSocket.recvfrom(bufferSize)expression =
```

```
    bytesAddressPair[0].decode()
```

```
    client_address = bytesAddressPair[1]
```

```
    num1, num2 = re.split('\+|-|\*|/|%', expression.replace(' ', ''))num1 = float(num1)
```

```
    num2 = float(num2)
```

```
    result = 0
```

```
    if '+' in expression:
```

```
        result = num1 + num2elif '-' in expression:
```

```
        result = num1 - num2elif '*' in expression:
```

```
        result = num1 * num2elif '/' in expression:
```

```
        if num2 == 0:
```

```
            result = 'Division by zero.'else:
```

```
            result = num1 / num2elif '%' in
```

```
expression:
```

```
    result = num1 % num2else:
```

```
    result = 'Invalid expression.'
```

```
print("Equation from client", client_address, ":", expression)print("Result to client:",
```

```
result)
```

```
UDPServerSocket.sendto(str(result).encode(), client_address)
```

```
UDPServerSocket.close()
```

#udp_client.py

```
import socket
```

```
expr = input("Enter an expression:")bytesToSend =
```

```
str.encode(expr)
```

```
serverAddressPort = ("127.0.0.1", 20001)
```

```
bufferSize = 1024
```

```
UDPClientSocket = socket.socket(family=socket.AF_INET, type=socket.SOCK_DGRAM)
```

```
UDPClientSocket.sendto(bytesToSend, serverAddressPort)
```

```
msgFromServer = UDPClientSocket.recvfrom(bufferSize)
```

```
print(expr, "=", msgFromServer[0].decode())
```

Thank You